

Fuzzing libpng 1.2.51 with AFL++: White-box and Binary-only Campaigns

Hassene Ezzeddine

Youssef Jeddi

Omar Bouden

Ismail Hammami

0 Introduction

libpng ships in nearly every browser and image toolkit and has a history of memory-safety CVEs, making it a representative fuzzing target. We target version 1.2.51.

1 Harness Design

Entry point : We target libpng's decoder, calling `png_read_info` → `three png_set_* transforms` → `png_read_update_info` → `png_read_image` → `png_read_end`. This is the API path every real consumer of untrusted PNGs traverses (browser decoders, ImageMagick...) so anything they parse from the network or a downloaded file ends up here, directly reachable from an attacker. It also concentrates libpng's richest parsing logic in one chain: chunk-header walking, zlib inflate of IDAT, scanline defiltering, 16 to 8-bit reduction etc. Historically, almost every libpng decoder CVE has lived in this path, including CVE-2015-8126 in `png_set_PLTE`, reached precisely through the `png_set_expand` call we enable.

Alternatives : We considered other entry points. The encoder (`png_write_*`) consumes structured pixel data the program already holds, not attacker-controlled bytes. Fuzzing the encoder would mean handing it random bytes as if they were pixels, but in real-world use no attacker ever gets to do that. So a bug in the decoder is directly reachable by sending a malicious PNG, whereas a bug in the encoder requires the attacker to already control the program's internal data, at which point the problem is bigger than a libpng issue. The encoder is still worth fuzzing in some cases, for example, a server that re-encodes user-uploaded images after resizing them, but we decided to focus on the decoding part only. We also thought of the simplified API (`png_image_*`) but it does not exist in 1.2.51.

Data flow : The harness passes mutated bytes through libpng's decode pipeline:

1. AFL writes a mutated file and passes its path as `argv[1]`.
2. `fopen + png_init_io` point libpng at the file.
3. `png_read_info` parses the 8-byte signature, IHDR. The `info` struct now holds dimensions, color type, bit depth.
4. The three `png_set_*` calls queue transforms to be applied

later, they don't read input. `png_read_update_info` recomputes the post-transform row size.

5. `png_read_image` is the main step: it consumes IDAT, decompresses via zlib, defilters scanlines, applies transforms, writes pixels into our row buffers.
6. `png_read_end` parses the IEND marker.

The persistent variant follows the same pipeline; the only difference is that bytes come from AFL's shared-memory buffer via a custom `mem_reader` callback plugged in with `png_set_read_fn` instead of `png_init_io`.

Guards : The `argc`, `fopen`, and `png_create_*` checks return 0 because they signal environment problems and crashing on any of them would create non-relevant findings.

The two guards that matter are the `setjmp` frame and the dimension cap. `setjmp` catches libpng's error path. Without it, the `longjmp` lands in an unwound frame and AFL records every malformed PNG as a crash. The dimension cap (4096×4096) prevents one-byte IHDR mutations from claiming large image dimensions, which would make libpng allocate gigabytes and OOM the process. It runs before `png_read_update_info`, where libpng performs that allocation.

2 Instrumentation and Sanitizers

2.1 Compiler and Toolchain

CC=afl-clang-fast. At every basic-block transition it inserts an edge-coverage update. After each execution, AFL++ knows exactly which edges were hit and can prioritise mutations that reach new ones. If omitted, AFL++ has no feedback and degrades to blind random fuzzing.

-fsanitize=address AddressSanitizer instruments every memory access. It must appear in `LDFLAGS` as well as `CFLAGS` so the linker pulls in the ASan runtime; omitting it from `LDFLAGS` produces a broken binary. Without ASan, AFL++ can only detect bugs that raise a signal. In our campaign, all crashes were ASan-raised aborts; without it we would have reported zero findings despite the bugs being present.

-g. Embeds debug symbols (function names, file names, line numbers). Not needed for fuzzing itself, but without it ASan crash traces show raw addresses instead of human-readable locations. Crash triage in Q5 would be significantly harder.

-O1. A deliberate middle ground. **-O0** produces code that behaves differently from production builds, making findings less credible. **-O2/-O3** aggressively inlines functions, collapses coverage map edges and makes stack traces less precise. **-O1** keeps light optimisation on, preserves recognisable stack frames and stays close enough to production for findings to be meaningful.

2.2 Configure Flag

--disable-shared. Produces only the static archive `libpng.a`, which is included directly into the harness binary at link time. Without it, `libpng` builds a shared library too, and at runtime the dynamic linker might load a system-installed `libpng.so` instead of our instrumented copy. Static linking eliminates this failure mode and makes the binary fully self-contained.

2.3 Linker Flags

-lpng12 -lz -lm. Link against `libpng 1.2` (the fuzz target), `zlib` (required for IDAT chunk decompression), and `libm` (used by `libpng`'s gamma correction math). Omitting any one of them breaks the link.

2.4 Three Separate libpng Builds

To be able to answer Q7 and Q8, we compiled `libpng` three times, changing exactly one variable at a time:

- **/install:** `afl-clang-fast` + ASan: main campaign build, used by `png_fuzz` and `png_fuzz_persistent`.
- **/install_noasan:** `afl-clang-fast`, no sanitizer: isolates ASan overhead for Q8 config 1.
- **/install_vanilla:** plain `gcc`, no instrumentation, no sanitizer: used by `png_fuzz_qemu` under `afl-fuzz -Q`.

Mixing instrumented and uninstrumented code in the QEMU binary would double-count edges; using the same build for Q8's exec-speed comparison would conflate two variables.

2.5 CRC Patch

We apply the AFL++ CRC-removal patch [1], which "neutralises" `png_crc_finish()` in `pngutil.c` so it always returns 0 regardless of checksum validity. Every PNG chunk ends with a CRC-32 computed over its type and data fields. When AFL++ mutates any byte inside a chunk, the checksum breaks. `libpng`'s default response is to call `png_error()` for critical chunks, the mutated data is rejected before any parsing code runs. For the other chunks, the data is silently discarded. In both cases, mutations never reach palette expansion, IDAT decompression and the code paths where all historical CVEs in `libpng` live. Without the patch, coverage saturates within

seconds at whatever the seed files already cover. With the patch, mutated chunk data flows straight to the chunk-specific parsers, and path discovery continues throughout the campaign.

3 Seed Corpus and Dictionary

Seeds: We generated six minimal valid PNGs with ImageMagick, so the corpus is fully reproducible. The set covers distinct combinations of color type, bit depth, and interlacing: `gray8.png` (color type 0, 8-bit), `gray16.png` (0/16), `rgb8.png` (2/8), `rgba8.png` (6/8), `palette.png` (3/8 with a PLTE chunk), and `interlaced.png` (RGB 8-bit with Adam7 interlacing). Each seed enters a *different* branch of the post-IHDR dispatch in `libpng`, so AFL++ begins calibration with broad initial coverage of the format-feature dispatch logic. Keeping every seed well under 1 KB also keeps calibration fast and per-input execution cheap.

Dictionary: We pass AFL++'s ready-made `dictionaries/png.dict` via `-x`. It contains the eight-byte PNG signature and the four-byte chunk-type identifiers (IHDR, PLTE, ...).

Formal interpretation: The dictionary entries are the terminal symbols of the PNG grammar: fixed chunk-type tokens (IHDR, IDAT, ...) that AFL++ can insert directly instead of synthesizing through random mutations.

Quantitative split: Table 4 reports the mutation strategies responsible for corpus growth during the 1-hour campaign. `havoc` dominated discovery with 839 generated paths (87.3% of all synthesized inputs), while deterministic stages (bit flips, arithmetic mutations, and insertion of "interesting" constants) together contributed a much smaller fraction. Dictionary-derived mutations contributed only a single new path, and `splice` contributed none.

This distribution matches the PNG format closely. Since every seed already contains the critical chunk-type tokens (IHDR, IDAT, PLTE, IEND), dictionary insertion has relatively low marginal utility. In contrast, small local mutations generated by `havoc` preserve enough global PNG structure for `libpng` to continue parsing deeply into the file.

`splice` performs poorly because PNG is a length-prefixed, chunk-structured format: concatenating fragments from two queue entries usually corrupts chunk boundaries and internal length fields even when the outer PNG signature remains valid.

4 Campaign Analysis

We analyze in depth the white-box instrumented campaign: `png_fuzz` (`afl-clang-fast` + AddressSanitizer, fork mode), run for one hour against `libpng 1.2.51` with AFL++ 4.41a.

Headline metrics: The four required metrics and other metrics are reported in Table 1 with more descriptions in Table 3. All values are read directly from `findings/default/fuzzer_stats` and can be verified

against the 1-hour AFL++ status panel in Figure 3.

Cumulative edge discovery: Figure 1 plots cumulative edges discovered over time, generated by `afl-plot` from `plot_data`. The curve follows the decelerating-S shape: a sharp initial rise, a knee phase, and a long flat tail. The curve rises sharply in the first 61 s (772 edges, 77.7% of the total), slows through a 30-minute knee phase, then flattens almost completely after $t \approx 1800$ s, with only six edges added in the final 30 minutes.

Saturation argument: Although `cycles_wo_finds=0` (every cycle, including the last, produced at least one new corpus entry), several observations support the claim that the campaign has reached *practical* saturation. The campaign added 772 edges in the first 60 s (≈ 0.08 s/edge) but only 6 edges in the last 1800 s (≈ 300 s/edge). In addition, **AFL is exploiting, not exploring**. At the snapshot, `pending_favs=0` and `pending_total=0` show AFL had finished cycle 57. The favored entries (156) and non-favored entries had all been visited in this cycle. Cycle 58 would necessarily revisit entries already mutated up to 57 times each, there is no fresh material in the queue. Furthermore, **33% is a harness ceiling, not a time ceiling**. The unreachable portion is discussed more in Q8.

5 Crash Triage

Campaign outcome. The 1-hour instrumented campaign produced 58 saved crashes and 0 hangs. AFL++ deduplicates crashes by coverage-bitmap fingerprint, so these 58 entries do not necessarily correspond to 58 distinct bugs. We therefore triaged one representative crash end-to-end and then clustered the remaining crashes by ASan summary.

We selected `findings/default/crashes/id:000000,sig:06,src:000000,time:2024,execs:5198,op:havoc,rep:2`, the earliest discovered crashing input.

Reproduction and minimization. The crash reproduces deterministically with: `ASAN_OPTIONS=symbolize=1:abort_on_error=0./png_fuzz $lang $crash\_file $rangle $, and we minimized it with: afl-tmin -i {crash_file} -o min.png - ./png_fuzz @@.`

The full stack trace is included in the appendix. The fault occurs inside `png_inflate()` in `pngutil.c`, part of libpng’s IDAT decompression pipeline.

The bug class is a *stack-use-after-scope*: a pointer derived from a stack-local object is dereferenced after the object’s lifetime has ended. This class of bug is typically invisible without AddressSanitizer because the memory remains mapped and often appears valid during normal execution. In practice, the bug manifests only because ASan poisons expired stack regions and converts the invalid access into a deterministic abort.

This path is highly security-relevant because every PNG decode processes IDAT data (reachably only because of the

CRC patch, §2.5).

Crash clustering. To determine whether the 58 crashes represented distinct bugs or many inputs reaching the same root cause, we replayed every crashing input and grouped them by ASan summary line:

```
for c in findings/default/crashes/id:*; do
  ASAN_OPTIONS=symbolize=1:abort_on_error=0 \
  ./png_fuzz "$c" 2>&1 \
  | grep -m1 '^SUMMARY: AddressSanitizer'
done | sort -u
```

All crashing inputs produced the same ASan summary, indicating a single underlying bug reached through many different mutation paths rather than dozens of unrelated vulnerabilities.

CVE mapping. We compared the stack trace against the public libpng CVE history and changelog entries for the 1.2.x series. We did not find a public CVE matching this exact `png_inflate()` stack-use-after-scope condition. We therefore report it as an ASan-confirmed memory-safety issue without assigning a CVE mapping.

6 Attack Surface Analysis

Mozilla Firefox: Firefox decodes every PNG a page references like (`<img>`, CSS backgrounds ...) through a libpng wrapper inside the sandboxed content process. A malicious `` on any ad-serving site triggers memory corruption (the stack-use-after-scope from §5); and combined with a sandbox-escape primitive this becomes a zero-interaction remote-code-execution chain. Our harness uses the pull reader (`png_read_image`), but Firefox uses the progressive push reader (`png_process_data`, in `pngpread.c`), which our harness never exercises.

ImageMagick: Web applications invoke ImageMagick to re-size and re-encode user uploads. Its PNG coder calls libpng for both reading and writing. An attacker submits a crafted PNG as a profile picture; the server’s resize pipeline decodes it, and the bug fires in a privileged `www-data` process. ImageMagick’s re-encode path also calls `png_write_*` in `pngutil.c` on attacker derived pixel data, a code path we never fuzz.

7 Binary-only Fuzzing in QEMU Mode

We ran a 38-minute campaign with `afl-fuzz -Q` on the vanilla `/png_fuzz_qemu` binary, using the same seeds and dictionary as the instrumented run. Both campaigns exceed the 30-min minimum. The three-axis comparison (Table 2), status screen (Figure 4) and `afl-plot` edges (Figure 2) curve are in the appendix.

QEMU’s higher exec speed looks paradoxical as dynamic translation should add overhead. But if we take ASan into consideration, the instrumented build carries an approximate 6× slowdown on libpng’s memory access while the QEMU

doesn't enable ASan, per §3.2. Holding the sanitizer constant, `/png_fuzz_noasan` runs at 14,170 execs/s versus QEMU's 3,322, i.e. compile-time instrumentation is $\sim 4.3\times$ faster than QEMU once the ASan confound is removed. The edges-discovered comparison needs a similar caveat. QEMU's `total_edges` is 65,536, the full AFL bitmap size, because QEMU has no compile-time edge count to report. The instrumented build has 2,982 total edges, the real CFG edges. So `bitmap_cvg` = 33.33 % (instrumented) and 4.27 % (QEMU) measure different denominators; only absolute `edges_found` are directly comparable.

QEMU runs the target binary through an emulator: it reads the binary's machine code in small chunks, translates each chunk into instructions the CPU can run, then executes them. It also inserts a small snippet that updates AFL's coverage map. The $\sim 4.3\times$ speedup measures the cost of dynamic translation. QEMU exists because it works on any binary, including closed-source ones where compile-time instrumentation isn't an option.

8 Instrumentation Depth and Performance

8.1 Edge counts: library vs. harness binary

To measure libpng's edge count in isolation we compiled a no-op stub linked against `libpng12.a` with `-whole-archive`, forcing the linker to include every object file in the archive regardless of reachability. Running AFL++ on this stub yields a total of **4 541 edges**.

This is the total number of instrumented edges libpng contributes when no dead-code elimination is applied: every encoder function, every function and every path is included. The 3 instrumented harness binaries report: **2 982 edges** for `/png_fuzz` (ASan + fork, main campaign), **2 997 edges** for `/png_fuzz_persistent` (ASan + persistent) and **2 976 edges** for `/png_fuzz_noasan` (no ASan + fork).

The difference $4541 - 2982 = 1559$ edges is explained entirely by linker dead-code elimination. When building `png_fuzz`, the linker performs reachability analysis from `main()` and drops every function not transitively called from the harness entry point. These 1 559 unreachable edges fall into 4 categories: the entire **encoder pipeline** (`png_write_*` functions); the **progressive/push reader** (`pngpread.c`) used by browsers but we never called it; **unenabled transformation functions** (`png_set_background`, etc.) which are only linked when explicitly called; and some **additional chunk handlers** for chunk types our seeds never represent. This 1 559-edge gap is the quantitative counterpart of Q6's missed attack surface: library code that exists but is structurally unreachable from our harness by construction.

8.2 Map density vs. total instrumented edges

Of the 2 982 edges present in the harness binary, the 1-hour campaign covered **994 edges**, **33.33% map density**, after 8.2 million executions and 57 corpus cycles. The remaining 1 988

edges were never hit. This 33.33% is a *saturation ceiling*, not a *time deficit*. The unreached edges fall into four categories:

- **Deep error-recovery branches:** libpng contains extensive defensive code for OOM fallbacks and impossible-state guards that normal or heavily mutated PNGs do not reach under our compile configuration.
- **Rare transform combinations:** the exact code path through `pngtran.c` depends on the combination of color type, bit depth, interlace setting, and active transform flags. Six seeds cover common tuples but not all; increasing coverage would require far more diverse inputs.
- **Chunks requiring specific internal structure:** handlers like `png_handle_iCCP` are reachable in principle but require a chunk with a valid embedded zlib body; random mutations rarely synthesize this by accident even with the dictionary.
- **Asymptotically unreachable code:** CRC-error handling edges are permanently dead because we patched out `png_crc_finish()`.

Running the campaign longer would not meaningfully change this figure. A different harness with more transformations enabled and a progressive-mode entry point would raise the ceiling, but that is a harness design limitation, not a campaign-time one.

8.3 Exec speed under three configurations

Table 5 summarises exec speed under the three configurations.

Config (2) → (1): removing ASan gives a $6.23\times$ speedup. ASan performs roughly two extra memory loads, a compare, and a branch per access. libpng's hot path is byte-level: filter reconstruction reads neighbouring scanline bytes, palette expansion performs indexed lookups, both call inner loops. This is exactly the access pattern ASan instruments most aggressively. The $6\times$ cost is notably worse than the typical " $2\times$ slowdown" of ASan. libpng has very little arithmetic and very many small memory accesses per input byte, making it a worst-case target for sanitizer overhead.

Config (2) → (3): switching fork to persistent mode gives a $5.1\times$ speedup ($9.4\times$ at steady state). In fork mode, AFL++ pays a full `fork()` cost per input. For a target that parses a small PNG in under a millisecond, this overhead is comparable to the actual parse time. Persistent mode forks once per 10 000 inputs, decreasing that fixed cost across thousands of iterations. The realised speedup is less than $10,000\times$ though because the harness still allocates and destroys memory on every iteration. The persistent mode campaign average (11 615) diverges from the steady-state figure ($\approx 21\,300$) because the slow warm-up seconds are averaged into a 60-second window that is too short for them to become negligible.

References

- [1] AFL++ Team. libpng crc removal patch, 2024.
[https://github.com/AFLplusplus/AFLplusplus/
tree/stable/utils/libpng_no_checksum](https://github.com/AFLplusplus/AFLplusplus/tree/stable/utils/libpng_no_checksum).

A Appendix

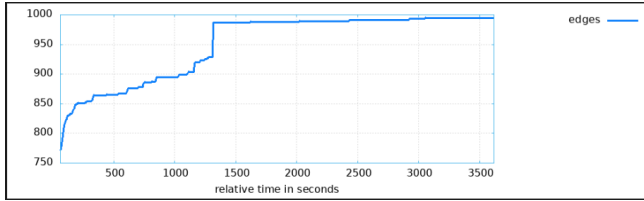


Figure 1: Instrumented campaign: afl-plot edges-discovered curve over the 1-hour run.

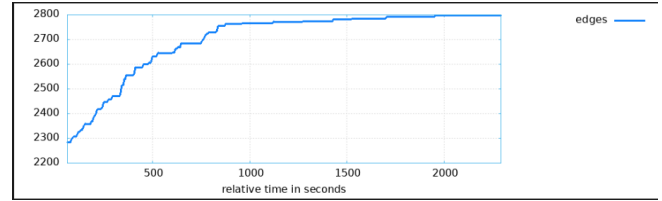


Figure 2: QEMU-mode campaign: afl-plot edges-discovered curve over the 38-min run.

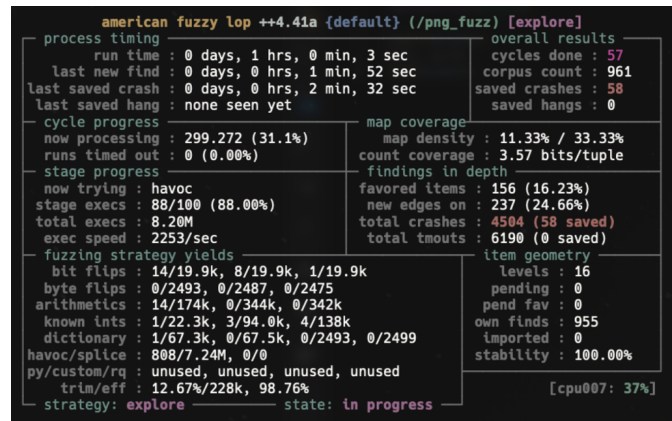


Figure 3: AFL++ status screen at the end of the 1-hour instrumented campaign (png_fuzz, afl-clang-fast + ASan, fork mode).

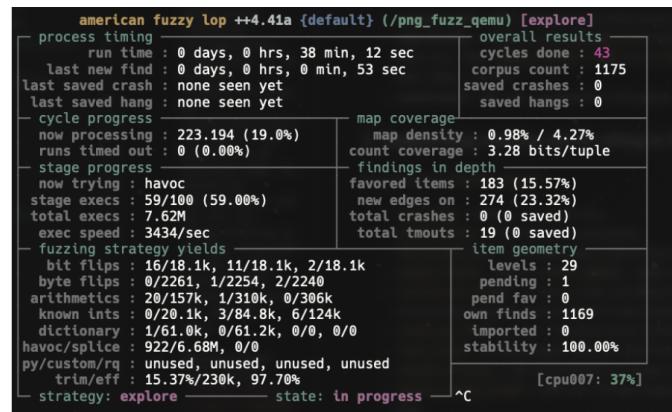


Figure 4: AFL++ status screen at the end of the QEMU-mode campaign.

Metric	Value
<i>Required</i>	
stability	100.00 %
corpus_count	961 (6 seeds + 955 finds)
bitmap_cvg (map density)	33.33 %
cycles_done	57
<i>Coverage breakdown</i>	
edges_found	994
total_edges	2982
unreachable edges (derived)	1988 (66.7 %)
<i>Queue composition</i>	
corpus_favored	156
non-favored entries (derived)	805
<i>Saturation signals</i>	
cycles_wo_finds	0
pending_favs	0
pending_total	0

Table 1: Campaign metrics for the 1-hour instrumented run (png_fuzz, afl-clang-fast + ASan, fork mode).

Axis	Instrumented	QEMU
Exec speed (execs/sec)	2,290	3,322
Edges discovered	989	2,797
Corpus count	922	1,175

Table 2: Three-axis comparison: instrumented vs QEMU-mode campaigns at the same wall-clock time. Instrumented = /png_fuzz (ASan + fork); QEMU = /png_fuzz_qemu (vanilla, with afl-fuzz -Q).

Metric	Description
Stability (100%)	Every input executed twice produces an identical coverage bitmap; the harness is fully deterministic.
Corpus count (961)	Six initial seeds plus 955 mutated descendants that each produced previously unseen coverage (new edge, or known in a new hit count bucket).
Map density (33.33%)	Fraction of edges in the instrumented binary that AFL reached at least once: edges_found/total_edges = 994/2982. The denominator includes edges in code paths that the harness cannot reach by construction (e.g., unenabled transforms).
Cycles done (57)	Number of complete passes AFL made over the corpus queue.

Table 3: AFL Fuzzing Statistics

Strategy	Paths	Share
havoc	839	87.3%
quick	31	3.2%
flip1/2/4	23	2.4%
arith8	14	1.5%
int8/16/32	8	0.8%
ext_UO	1	0.1%
splice	0	0.0%

Table 4: Corpus-growth contribution by AFL++ mutation strategy after the 1-hour instrumented campaign (961 total queue entries; 6 seeds + 955 generated inputs).

Config	Description	exec/sec	edges found
(1)	no-ASan + fork	14 170	821 / 2 976
(2)	ASan + fork (1h run)	2 274	994 / 2 982
(3)	ASan + persistent	11 615 [†]	837 / 2 997

[†]60-second average; steady-state $\approx 21\,300$ exec/sec.

Table 5: Exec speed and edge coverage under three harness configurations.